

Add New Target Detector: Integration Report

This report explains how to add a new object detection model to AdvReal, including the API wrapper, configuration, and training-path hooks.

1) Where detectors are wired in

- Detector registry and initialization: `detlib/utils.py`
- Base detector interface (required methods): `detlib/base.py`
- Attack loop consumes detector outputs: `attack/methods/base.py`
- Multi-detector orchestration: `attack/attacker.py`
- 3D/patch training path uses model + extractor: `train.py` and `load_data.py`
- Output conversion helpers for some models: `utils_camou.py`

2) Detector output contract (required)

Your detector wrapper must return a dict with these keys:

```
{
  "bbox_array": List[Tensor[N, 6]], # per-image, [x1, y1, x2, y2, conf,
  cls_id], normalized to [0,1]
  "obj_confs": Tensor[B, K],        # per-image confidence/objectness used
  for gradients
  "cls_max_ids": Optional[Tensor[B, K]] # class indices aligned with
  obj_confs (can be None if unused)
}
```

Notes:

- `bbox_array` coordinates must be normalized to `[0, 1]` (divide by image width/height).
- `obj_confs` must be a torch tensor on the same device as the detector output.
- `cls_max_ids` is required only if you enable class-specific loss selection in the attacker.

3) Steps to add a detector (multi-detector path)

Step A: Create a wrapper class

Create a new file under `detlib/HHDet/<your_model>/api.py` and implement `DetectorBase`.

Minimum methods:

- **init(...)** to set device, sizes, thresholds
- **load(...)** to build the model and load weights
- **call(...)** to run inference and build the output dict

Skeleton:

```
import torch
from ...base import DetectorBase

class HMyDetector(DetectorBase):
    def __init__(self, name, cfg, input_tensor_size=640,
device=torch.device("cuda:0" if torch.cuda.is_available() else "cpu")):
        super().__init__(name, cfg, input_tensor_size, device)
        # set any model-specific fields here

    def load(self, model_weights, **args):
        self.detector = ...
        self.detector.to(self.device)
        self.eval()

    def __call__(self, batch_tensor, **kwargs):
        # 1) run model
        # 2) produce bbox_array list (Nx6 per image)
        # 3) produce obj_confs tensor
        # 4) optional cls_max_ids
        return {
            "bbox_array": bbox_array,
            "obj_confs": obj_confs,
            "cls_max_ids": cls_max_ids
        }
```

Step B: Register the detector in init_detector

Add a new branch in detlib/utils.py:

```
elif detector_name == "mydet":
    from detlib.HHDet import HMyDetector
    detector = HMyDetector(name=detector_name, cfg=cfg, device=device)
    detector.load(model_weights=..., model_config=...)
```

Step C: Add a baseline config

Add configs/baseline/mydet.yaml with the detector name and thresholds:

```
DETECTOR:
  NAME: ["MYDET"]
  INPUT_SIZE: [416, 416]
  BATCH_SIZE: 4
  CONF_THRESH: 0.5
  IOU_THRESH: 0.45
```

Step D: Place weights and config

Follow existing layout for YOLOv5/YOLOv11 in `detlib/HHDet//weights/`. Adjust paths in `detlib/utils.py`.

4) Steps to add a detector (train.py path)

This is needed if you want to use `train.py` with `args.arch`.

Step A: Add model branch in PatchTrainer

In `train.py`, add a new `"elif args.arch == ..."` block to load the model and freeze params.

Step B: Add a probability extractor

Add a new extractor class in `load_data.py` similar to `YOLOv5MaxProbExtractor`.

- Convert model output to a standard box format
- Compute IoU with ground truth and select scores

Step C: Add output conversion if needed

If your model output format is unique, extend `utils_camou.get_region_boxes_general` to translate it into:

```
[x_center, y_center, width, height, obj_conf, cls_conf, cls_id]
```

Step D: Wire the extractor in train.py

Add a new branch in the extractor selection (same block as YOLOv5/YOLOv11).

5) Step-by-step example: add YOLOv11 (beginner walk-through)

This example mirrors the existing YOLOv11 integration and can be used as a template for adding a new model.

Step 1: Create the wrapper (model API)

- File location: `detlib/HHDet/yolov11/api.py`
- Goal: implement `DetectorBase` so the attacker can call the model and receive the expected output dict.

Key points from the existing wrapper:

- The model is loaded with the Ultralytics API, then the underlying PyTorch model is used.
- The forward pass returns raw predictions and a normalized bbox list.
- `obj_confs` are computed from class logits to keep gradients flowing.

Step 2: Register the name in init_detector

- File: `detlib/utils.py`
- Branch name must be lowercase, so "YOLOV11" in config becomes "yolov11" in code.

Example branch (already in the repo):

```
elif detector_name == "yolov11":
    from detlib.HHDet import HHYolov11
    detector = HHYolov11(name=detector_name, cfg=cfg, device=device)
    weights_path = os.path.join(DET_LIB,
                                "HHDet/yolov11/weights/yolo11s.pt")
    detector.load(model_weights=weights_path)
```

Step 3: Add a baseline config

- File: configs/baseline/v11.yaml
- Keep thresholds and input size consistent with the rest of the project.

Minimal config:

```
DETECTOR:
  NAME: ["YOLOV11"]
  INPUT_SIZE: [416, 416]
  BATCH_SIZE: 8
  CONF_THRESH: 0.5
  IOU_THRESH: 0.45
```

Step 4: Place weights

- Put the weights file at: detlib/HHDet/yolov11/weights/yolo11s.pt
- If you name the file differently, update the path in detlib/utils.py.

Step 5: (Optional) Enable training path in train.py

If you want to use the 3D/patch training path:

1. Add model selection in train.py under args.arch == "yolov11" (already present).
2. Ensure the probability extractor exists in load_data.py (YOLOv11MaxProbExtractor is already present).
3. Confirm utils_camou.get_region_boxes_general has a yolov11 branch (already present).

Step 6: Quick sanity check

- Create a config file with DETECTOR.NAME: ["YOLOV11"]
- Run a small inference loop through UniversalAttacker.detect_bbox
- Verify bbox_array values are in [0, 1] and obj_confs is a tensor

6) Validation checklist

- Wrapper returns bbox_array normalized to [0, 1]
- obj_confs is a torch tensor on the correct device
- Config DETECTOR.NAME matches init_detector branch
- Weights path is correct and readable

- For train.py path, extractor can parse model outputs and compute IoU

7) Common pitfalls

- Forgetting to normalize bbox coordinates
- Returning a list instead of a tensor for obj_confs
- Model outputs on CPU while attack runs on GPU
- Config path pointing to machine-specific directories

8) Example: where to look for references

- detlib/utils.py (detector registry)
- detlib/base.py (required interface)
- detlib/HHDet/yolov5/api.py (wrapper example)
- detlib/HHDet/yolov11/api.py (wrapper example)
- load_data.py (probability extractors)
- utils_camou.py (output conversion helpers)
- train.py (arch selection and extractor binding)